

Migração para Clean Architecture e Injeção de Dependência

O que mudou no backend do PDL PRO, por que mudou,
e como a arquitetura em camadas passa a guiar novas features.

Backend · Django/DRF · Domain · Application · Infrastructure · Presentation

Setembro 2026 · Suíte validada: 758 testes

Propósito

Este documento registra a conclusão da migração arquitetural do backend: regras de negócio desacopladas de Django/ORM, casos de uso orquestrados por portas, infraestrutura como adaptador e presentation fina. O objetivo não foi cerimônia — foi tornar o sistema testável, substituível nas bordas e consistente entre apps.

1. Por que migrar

Antes da migração, partes relevantes do backend misturavam transporte HTTP, regras de negócio e acesso a banco na mesma pilha. Views e serviços conheciam ``.objects``, ``transaction.atomic`` e detalhes de modelos Django. Isso tornava difícil testar a regra sem subir o ORM, trocar um adaptador (ex.: gateway Lineage) sem arrastar a application, e manter o mesmo padrão entre wallet, shop, games e accounts.

Clean Architecture + injeção de dependência resolvem o problema pela direção das dependências: o domínio não conhece Django; a application só fala com portas; a infrastructure implementa essas portas; a presentation traduz HTTP. DI fecha o ciclo registrando adaptadores uma vez por app e resolvendo casos de uso por construtor tipado.

Motivo	Efeito prático
Testabilidade	Regras exercitadas com fakes nas portas, sem banco real.
Substituibilidade	Trocar ORM/gateway/provedor sem reescrever casos de uso.
Consistência	Mesmo desenho em todos os apps de negócio.
Segurança de borda	Autorização, dinheiro e idempotência ficam na application/domain.
Manutenção	Views finas; mudanças de contrato HTTP não espalham regra.

2. Modelo alvo

A regra de dependência adotada no repositório:

presentation → application → domain

infrastructure → domain (implementa as portas)

Application e presentation não importam infrastructure nem usam ORM.

Presentation	HTTP / WebSocket / cookies
Application	Casos de uso e orquestração
Domain	Regras, portas e exceções
Infrastructure	ORM, gateways, providers

Figura — camadas e responsabilidade. A seta de dependência aponta para dentro, em direção ao domínio.

Camada	Pasta	Responsabilidade
Domínio	domain/	Entidades, políticas, exceções e interfaces (portas).
Aplicação	application/	Casos de uso, inputs tipados e orquestração.
Infraestrutura	infrastructure/	ORM, repositórios, gateways, providers DI.
Apresentação	presentation/	Views, serializers, URLs, cookies HTTP.

common/

common/ permanece transversal: container DI, UnitOfWork, contrato de erro, paginação, permissões e OpenAPI. Não deve absorver regra de negócio de um app.

3. Injeção de dependência

Cada app de negócio registra um `AppProvider` no `AppConfig.ready()`. O catálogo compõe o container raiz. O middleware cria um escopo por requisição; views `InjectedAPIView` resolvem o caso de uso com `self.resolve(...)`.

Lifetime	Uso
SINGLETON	Objetos imutáveis / pools no processo.
SCOPED	Uma instância por requisição (repositórios, políticas).
TRANSIENT	Nova instância a cada resolve (casos de uso).

Comandos de management e tasks também passam pelo container (`DependencyInjection.root().create_scope()`), em vez de instanciar serviços concretos na mão.

Decisão: sem locator na application

Service locator como fallback dentro de casos de uso foi eliminado. Dependências entram pelo construtor tipado — o container inspeciona e injeta. Isso evita acoplamento escondido e falhas só em runtime de produção.

4. O que foi migrado

A migração foi feita em ondas, app por app, sempre com testes no mesmo conjunto de alterações. Resumo das frentes:

Frente	Resultado
Wallet / games / paid services	Crédito, débito e recompensas via <code>IWalletRepository</code> ; settle e grant com portas.
Shop / payment	Checkout, webhooks e listagens por use cases; gateways mockáveis na borda externa.
Accounts	Auth, sessões, 2FA, OAuth, WebAuthn, e-mail e progresso via portas + DI.
Programs / support / themes	Providers, use cases e views finas; staff consome portas dos apps donos.
Server / inventory / auction / marketplace	Gateway Lineage e operações de personagem/itens atrás de portas.
Content	Catálogo, Denkynho, wardrobe e FAQ com injeção; chat/assistant sem ORM direto.
Purity pass	Sem ORM em application/presentation; sem cycle infra→presentation; seed via porta.

5. Decisões-chave e por quê

5.1 Portas em vez de models nas views

Views deixaram de chamar `.objects` ou `ModelSerializer` preso a `models` de infraestrutura. Elas validam o transporte, resolvem o use case e serializam o resultado. Motivo: HTTP muda com frequência; a regra não deve acompanhar cada detalhe de serializer.

5.2 UnitOfWork no lugar de atomic espalhado

`transaction.atomic` saiu da application. Transações ficam no adaptador / `UnitOfWork`, preservando rollback em fluxos de dinheiro, itens e recompensas sem vazar Django para a camada de orquestração.

5.3 Staff consome portas dos apps donos

Endpoints operacionais não reimplementam regra de `wallet/games/server`. Staff resolve portas admin já registradas pelos providers donos. Motivo: uma política, dois canais (customer vs staff), menos duplicação e menos drift.

5.4 Cookies JWT: quem monta a Response

Residual consciente fechado: a infraestrutura não monta mais `Response/DRF`. `IAuthSessionService.require_user` materializa o usuário necessário a `RefreshToken.for_user`; `build_auth_response` e helpers de cookie ficam na presentation. Motivo: HTTP é transporte — não pode puxar a infra para cima.

5.5 Seed de preview via porta

O comando `seed_program_preview` só checa o ambiente preview e resolve `IPreviewSeedService`. O ORM vive em `DjangoPreviewSeedService`. Motivo: management commands são borda de infra, mas o padrão DI permanece; a regra de “quem usa ORM” fica explícita no adaptador.

5.6 Relação ORM não vaza pela application

Caminhos do tipo “andar relações Django” foram empurrados para repositórios (ex.: dumps tipados). A application recebe dados já materializados pelas portas. Motivo: lazy loading e queries escondidas destroem previsibilidade e testes.

6. Antes × depois

Antes	Depois
View com regra + ORM	View fina → UseCase → Porta → Adapter
Service locator opcional	Construtor tipado obrigatório
atomic na application	UnitOfWork / adaptador
AuthSession montava Response	require_user + cookies na presentation
Seed com ORM no comando	Comando DI + adaptador ORM
Duplicação staff × customer	Portas compartilhadas nos apps donos

7. Cobertura por módulo

App	Papel na arquitetura
accounts	Usuários, auth, 2FA, OAuth, WebAuthn, progresso
server	Status, personagens, rankings, ILineageGateway
wallet	Saldo, transferências, bônus
shop	Catálogo, carrinho, checkout
payment	Pedidos, provedores, webhooks
inventory	Depósito, retirada, troca
marketplace / auction	Anúncios, compra, leilões
games	Minigames, economia, battle pass, daily/fishing
content	Notícias, FAQ, Denkynho, wardrobe
communication	Notificações e push
programs	Apoiadores, roadmap, seed preview
support / themes / staff	Chamados, temas, operação

Todos esses apps possuem AppProvider registrando portas, adaptadores e casos de uso com lifetimes adequados.

8. Como implementar uma feature daqui pra frente

- Modele entidade, exceção ou interface no domínio.
- Crie o caso de uso com input tipado na application.
- Implemente repositório/gateway na infrastructure.
- Registre interface + implementação + use case no provider.
- Exponha serializer/view fina e rota no namespace certo.
- No frontend, concentre HTTP em services de domínio + `http.ts`.
- Cubra sucesso, inválidos, authz, limites e idempotência com testes.
- Atualize OpenAPI/docs quando o contrato público mudar.

Contrato de API

Namespaces: `auth` · `public` · `shared` · `customer` · `staff` · `system`. Escolha o namespace pelo ator e pelo risco — não pela conveniência de URL.

9. Validação e estado atual

A suíte de backend sob apps/ fechou em 758 testes passando após o fechamento dos residuais. Critérios de auditoria confirmados na varredura final:

- Domínio sem imports Django/DRF/infra.
- Application sem `.objects`, sem `atomic`, sem `import` de infrastructure.
- Presentation sem ORM; middleware de catálogo ativo vive em infrastructure.
- Management commands sem ORM direto (seed via porta).

- Sem dependência infrastructure → presentation.

100% do escopo migrado

Atritos de borda aceitos (não reabrem o escopo): parsing JWT com simplejwt em sessions da application; usuário ORM no limite `RefreshToken.for_user`; helpers de nome de cookie nas camadas de transporte. São fricções conscientes de biblioteca/HTTP, não regressão de Clean Architecture.

10. Conclusão

A migração transformou o backend do PDL PRO de um conjunto de fluxos Django-cêntricos em um sistema modular com regra de dependência explícita. O ganho imediato é previsibilidade: cada feature nova tem um caminho óbvio (porta → use case → adapter → view). O ganho estrutural é longevidade: testes, troca de integrações e revisão de segurança passam a operar em bordas estáveis.

Documentação viva: `docs/arquitetura/visao-geral.md`, `apps.md`, `common.md` e `reutilizacao.md`. Este PDF é o registro narrativo da migração concluída em setembro de 2026.

PDL PRO — arquitetura em camadas com DI
Migrado · Validado · Pronto para novas features